

VIT-AP
UNIVERSITY

DIGITAL LOGIC DESIGN
(Course Code: ECE 1003)

Module-1:Lecture-12
Boolean Algebra

CONTENTS

Module-1 (Part-2)

- ❖ Logic gates and universal gates
- ❖ Tri-state logic
- ❖ Boolean algebra
- ❖ SOP and POS minimization
- ❖ **K-maps upto 4-variables**
- ❖ **Don't care logic**

CONTENTS

Lecture-12

- ❖ Gate Level Minimization
 - 4 Variable K-map
 - Don't care conditions

BOOKS

Textbooks

1. M.Morris Mano, Michael D Ciletti, **Digital Design**, 5th edition, Pearson Publishers, 2013.
2. R.P. Jain, “**Modern Digital Electronics**”, 4th edition, TMH.

References

1. M.Morris Mano, Charles R. Kime, Tom Martin, **Logic and Computer Design Fundamentals**, 4th edition, Pearson Publishers.
2. C. H. Roth and L. L. Kinney, **Fundamentals of Logic Design**, 5th edition, Cengage Publishers.

Gate Level Minimization

4-VARIABLE K-MAP

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

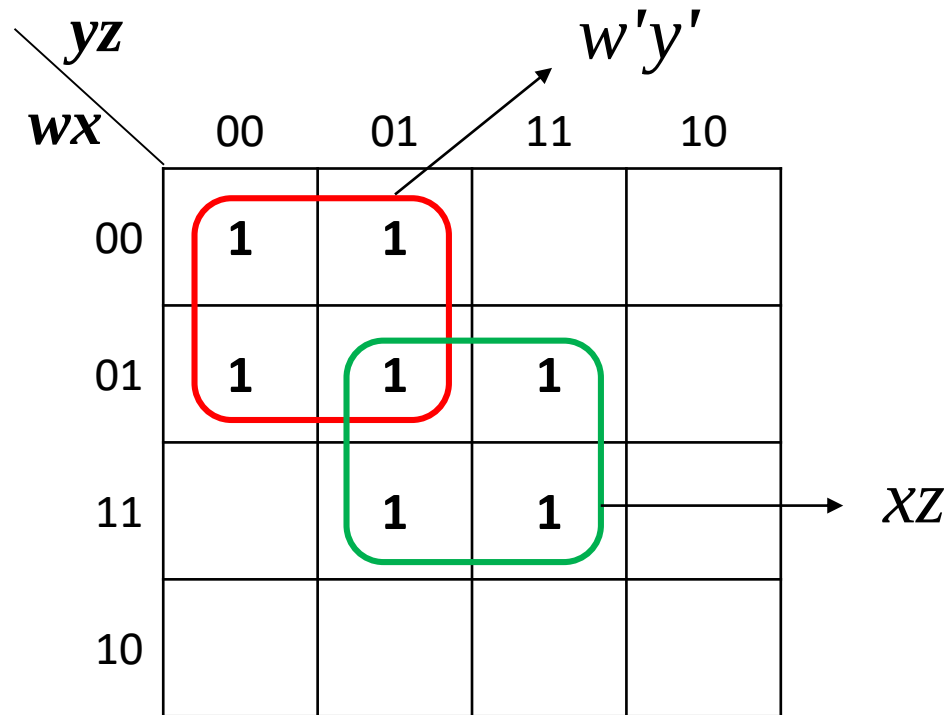
		y				
		yz	00	01	11	10
w	00	m_0	$w'x'y'z'$	$w'x'y'z$	$w'x'yz$	$w'x'yz'$
		m_4	$w'xy'z'$	$w'xy'z$	$w'xyz$	$w'xyz'$
	01	m_5	$w'xy'z$	$w'xyz$	$w'xyz'$	$w'xyz$
		m_7	$w'xyz$	$w'xyz'$	$w'xyz$	$w'xyz'$
11	11	m_{12}	$wxy'z'$	$wxy'z$	$wxyz$	$wxyz'$
		m_{13}	$wxy'z$	$wxyz$	$wxyz'$	$wxyz$
10	10	m_{15}	$wxyz$	$wxyz'$	$wxyz$	$wxyz'$
		m_{14}	$wxyz'$	$wxyz$	$wxyz'$	$wxyz$
x	11	m_8	$wx'y'z'$	$wx'y'z$	$wx'yz$	$wx'yz'$
		m_9	$wx'y'z$	$wx'yz$	$wx'yz'$	$wx'yz$
	10	m_{11}	$wx'yz$	$wx'yz'$	$wx'yz$	$wx'yz'$
		m_{10}	$wx'yz'$	$wx'yz$	$wx'yz'$	$wx'yz$
		z				

4-VARIABLE K-MAP

- ✓ **One square** represents one minterm, giving a term with **four literals**.
- ✓ **Two adjacent squares** represent a term with **three literals**, i.e., differed by one literal.
- ✓ **Four adjacent squares** represent a term with **two literals**, i.e., differed by two literals.
- ✓ **Eight adjacent squares** represent a term with **one literal**, i.e., differed by three literals.
- ✓ **Sixteen adjacent squares** produce a function that is always equal to **1**, i.e., differed by all four literals.

4-VARIABLE K-MAP

Example: Simplify $F = \Sigma(0, 1, 4, 5, 7, 13, 15)$



$$F = w'y' + xz$$

4-VARIABLE K-MAP

Note: In the problem shown below, the 1's present form a quad.

CD					
AB		00	01	11	10
00	1				1
01					
11					
10	1				1

$B'D'$

$$A'B'C'D' + A'B'CD' = A'B'D'$$

$$AB'C'D' + AB'CD' = AB'D'$$

$$A'B'D' + AB'D' = B'D'$$

4-VARIABLE K-MAP

Example: Simplify $F = \Sigma(0, 2, 3, 4, 5, 6, 7, 8, 10, 11, 15)$

$AB \backslash CD$		CD			
		00	01	11	10
00	1			1	1
01	1	1	1	1	1
11				1	
10	1			1	1

Diagram illustrating the 4-variable K-map for the function $F = \Sigma(0, 2, 3, 4, 5, 6, 7, 8, 10, 11, 15)$. The map is a 4x4 grid with rows labeled AB (00, 01, 11, 10) and columns labeled CD (00, 01, 11, 10). The cells containing 1s are: (00,00), (00,11), (00,10), (01,00), (01,01), (01,11), (01,10), (11,11), (10,00), (10,11), and (10,10). The map shows three prime implicants: a blue rectangle covering the top row (00) and the second row (01) for columns 00, 11, and 10, labeled $A'B$; a green rectangle covering the first and fourth rows (00 and 10) for columns 11 and 10, labeled CD ; and a red rectangle covering the first and fourth rows (00 and 10) for columns 00 and 11, labeled $B'D'$.

$$F = A'B + CD + B'D'$$

4-VARIABLE K-MAP

Example: Simplify the Boolean function using K map,

$$F = A'B'C' + B'CD' + A'BCD' + AB'C'$$

Solution:

$$F = A'B'C' + B'CD' + A'BCD' + AB'C'$$

$$= A'B'C'(D + D') + (A + A')B'CD' + A'BCD' + AB'C'(D + D')$$

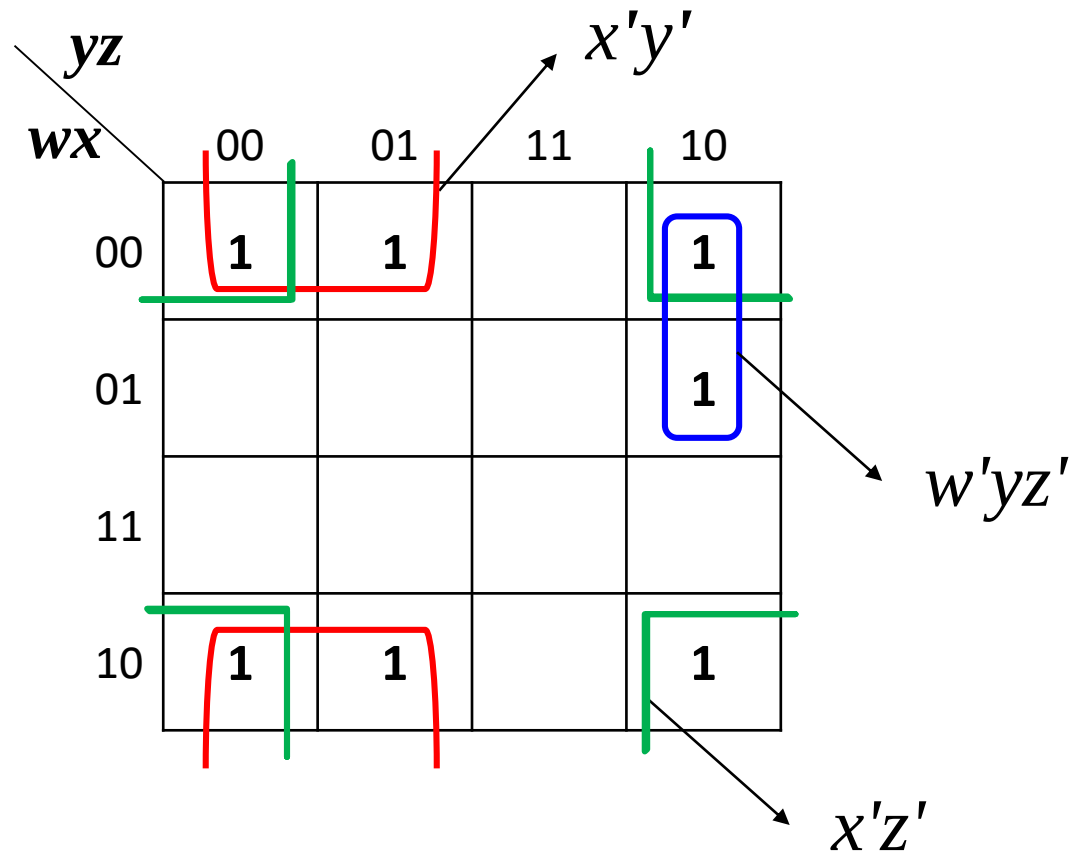
$$= A'B'C'D + A'B'C'D' + AB'CD' + A'B'CD' + A'BCD' + AB'C'D + AB'C'D'$$

$$= m_1 + m_0 + m_{10} + m_2 + m_6 + m_9 + m_8$$

$$= \Sigma(0, 1, 2, 6, 8, 9, 10)$$

4-VARIABLE K-MAP

Example: Simplify $F = \Sigma(0, 1, 2, 6, 8, 9, 10)$



$$F = x'y' + x'z' + w'yz'$$

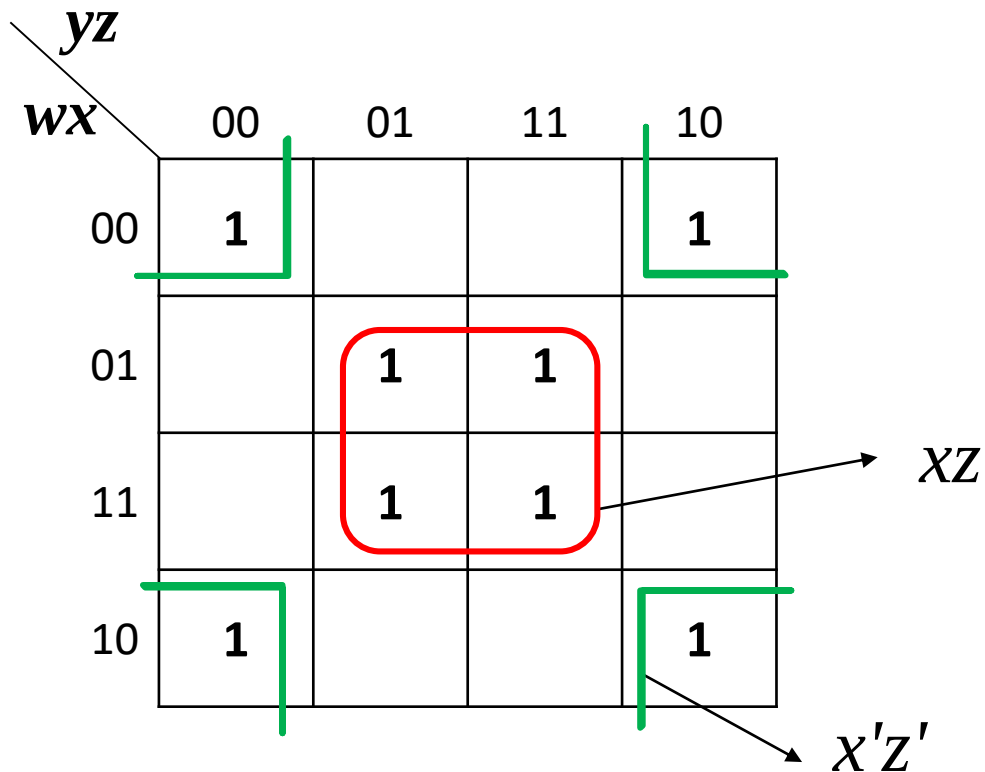
4-VARIABLE K-MAP

Prime Implicants & Essential Prime Implicants

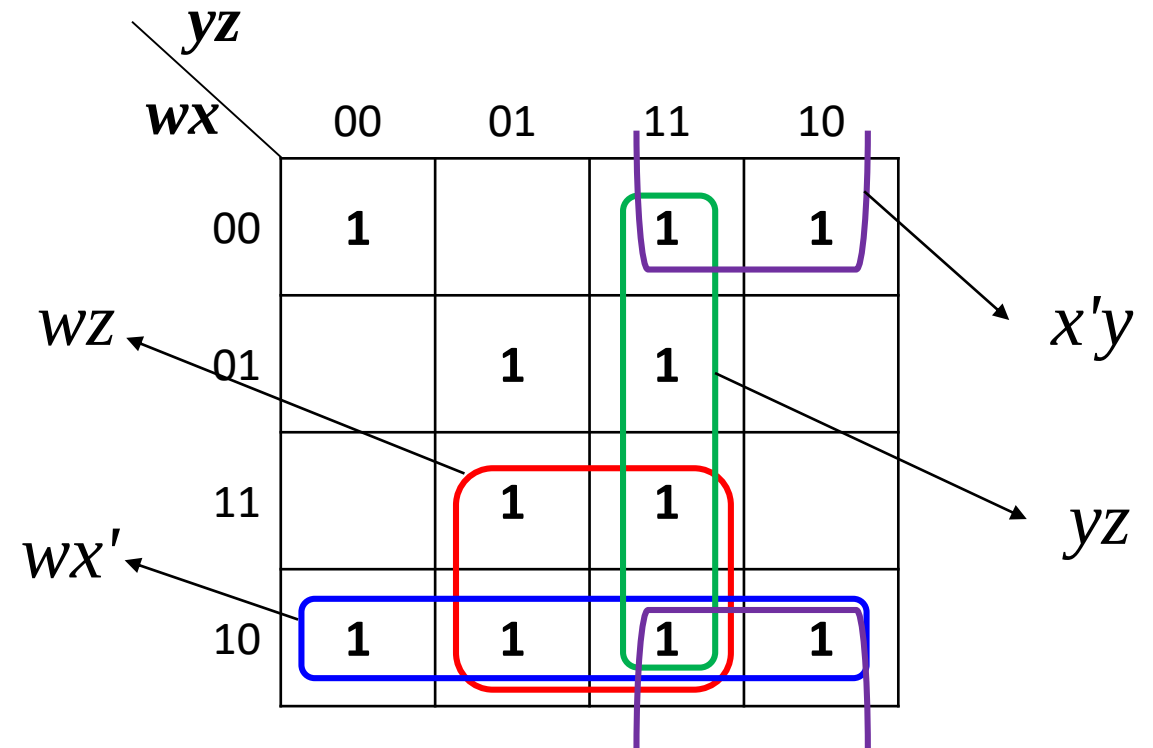
- ✓ A **prime implicant** is a product term obtained by combining the maximum possible number of adjacent squares in the map. If a minterm in a square is covered by only **one prime implicant**, that prime implicant is said to be **essential**.
- ✓ This means that a single 1 on a map represents a prime implicant if it is not adjacent to any other 1's.
- ✓ Two adjacent 1's form a prime implicant, provided that they are not within a group of four adjacent squares.
- ✓ Four adjacent 1's form a prime implicant if they are not within a group of eight adjacent squares, and so on.
- ✓ The **essential prime implicants** are found by looking at each square marked with a 1 and checking the number of prime implicants that cover it. The prime implicant is essential if it is the only prime implicant that covers the minterm.

4-VARIABLE K-MAP

Example-1: Simplify $F = \Sigma(0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$



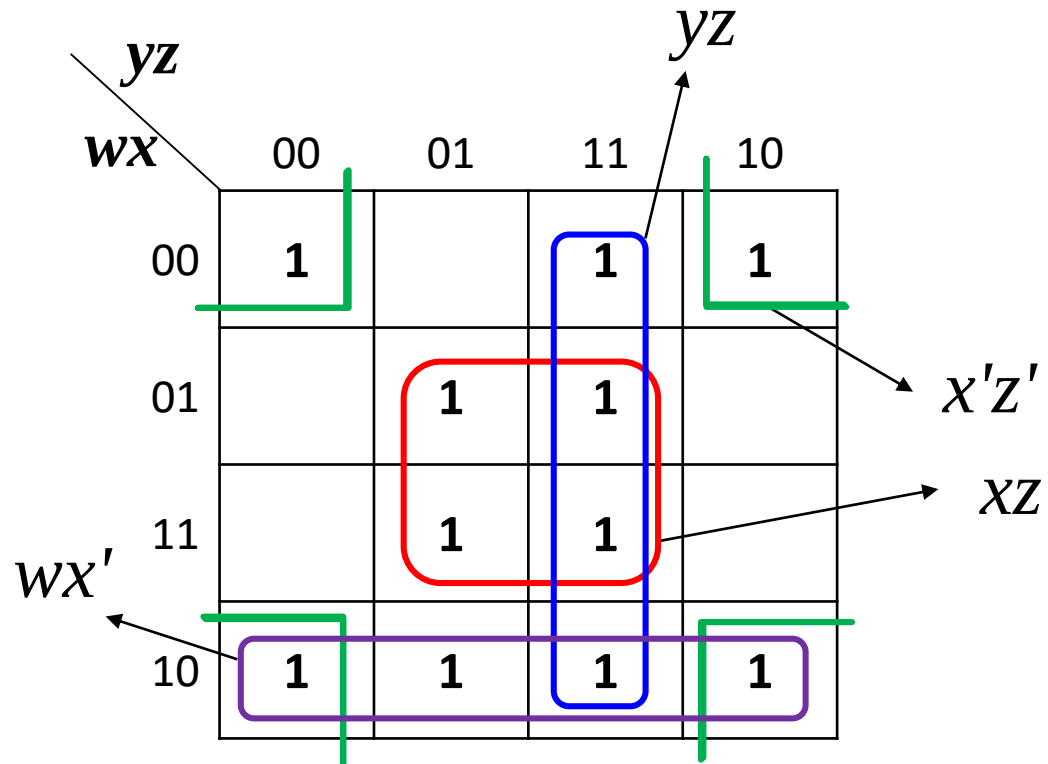
Essential prime implicants: $xz, x'z'$



Prime implicants: $wz, wx', yz, x'y$

4-VARIABLE K-MAP

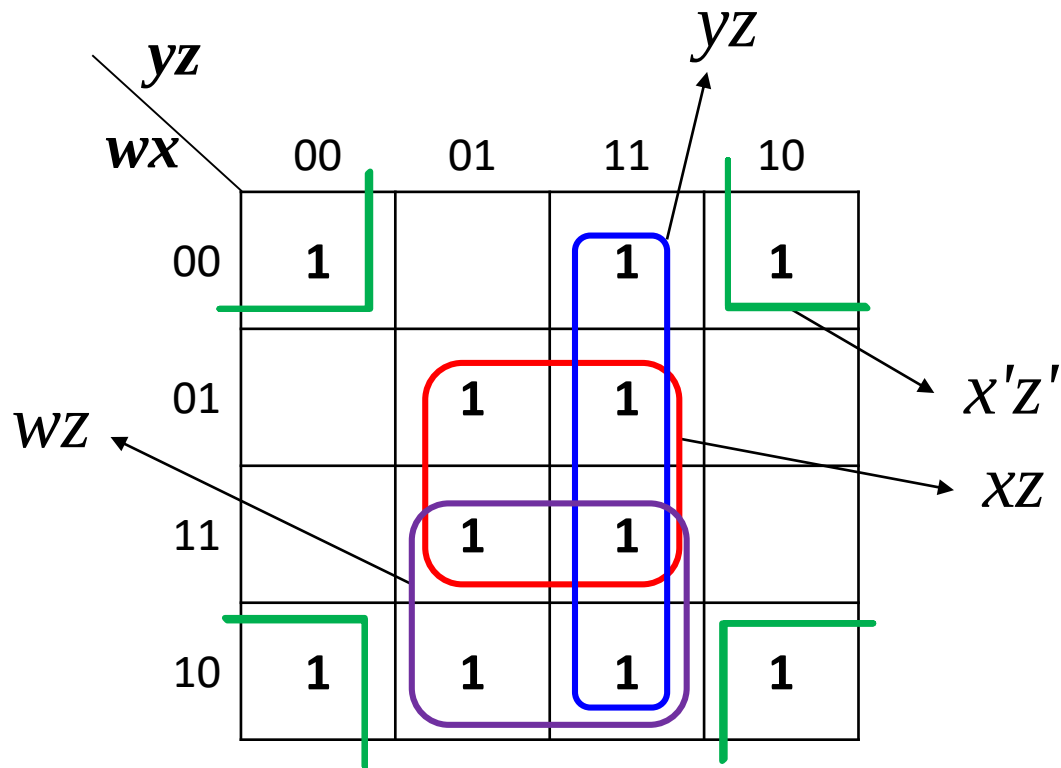
Example-1(cont...): One possible combination is given below



$$F = xz + x'z' + yz + wx'$$

4-VARIABLE K-MAP

Example-1(cont...): Another possible combination is given below



$$F = xz + x'z' + yz + wz$$

- ✓ Other possible results can be obtained as

$$F = xz + x'z' + x'y + wz$$

$$F = xz + x'z' + x'y + wx'$$

- ✓ Here, essential prime implicants are used in each result.
- ✓ However, maximum two prime implicants are required in each result.

DON'T CARE CONDITION

- ✓ Sometimes, Boolean functions contain some **unspecified minterms** called as incomplete specified functions termed **don't care conditions**.
- ✓ These don't-care conditions can be used on a map to provide **further simplification of the Boolean expression**.
- ✓ When simplifying the function, we can choose to include each **don't-care minterm with either the 1's or the 0's**, depending on which combination gives the simplest expression.
- ✓ E.g., 4-bit binary decimal digits has 6 combinations that are unused (i.e. from 1010 to 1111).
- ✓ To distinguish the don't-care condition from 1's and 0's, an **X** is used.
- ✓ Thus, an X inside a square in the map indicates that we don't care whether the value of 0 or 1 is assigned to F for the particular minterm.

DON'T CARE CONDITION

Example: Simplify the Boolean function $F = \Sigma(0, 1, 3, 7, 11, 15)$
which has the don't care conditions $d = \Sigma(2, 5, 6, 13)$

CD		AB			
		00	01	11	10
00		1	1	1	X
01			X	1	X
11			X	1	
10				1	

$A'B'$

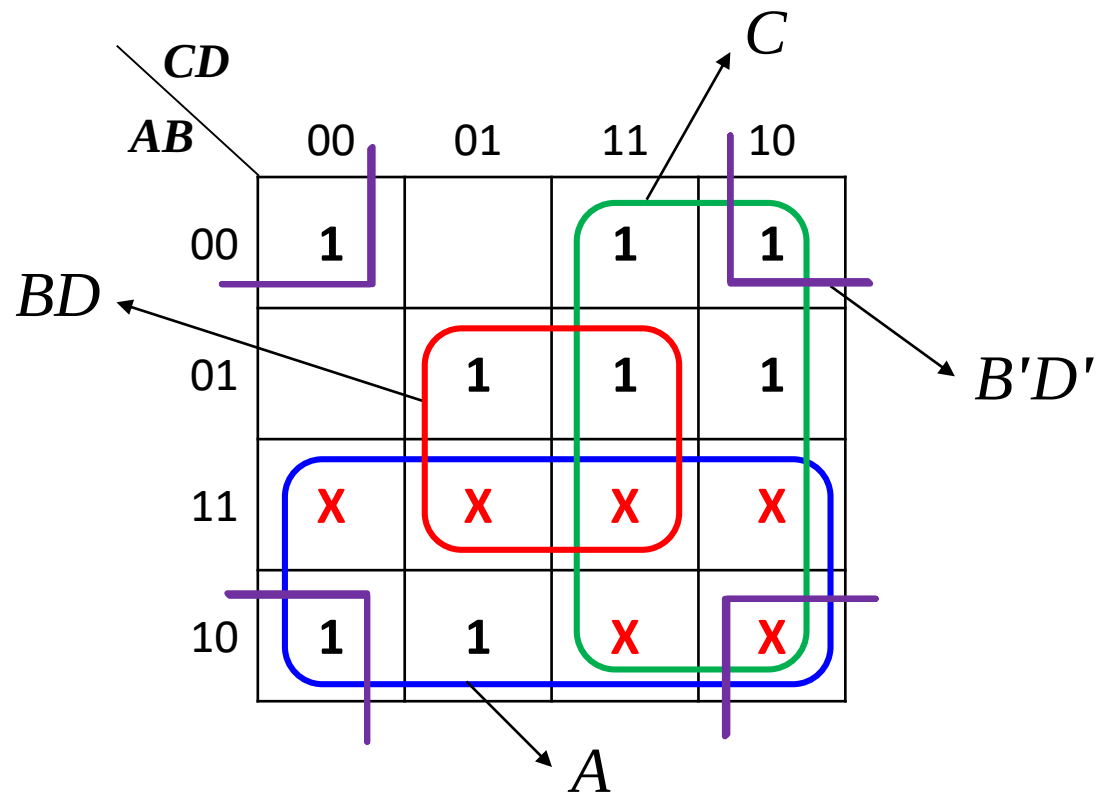
CD

$$F = A'B' + CD$$

DON'T CARE CONDITION

Example: Simplify the Boolean function

$$F = \Sigma(0, 2, 3, 5, 6, 7, 8, 9) + d(10, 11, 12, 13, 14, 15)$$



$$F = A + C + B'D' + BD$$

DON'T CARE CONDITION

Example: Simplify the Boolean function $F = \Sigma(1, 3, 7, 11, 15) + d(0, 2, 5)$

		y				
		yz	00	01	11	10
w	$w'x'$	00	m_0 X	m_1 1	m_3 1	m_2 X
	01	m_4 0	m_5 X	m_7 1	m_6 0	
	11	m_{12} 0	m_{13} 0	m_{15} 1	m_{14} 0	
	10	m_8 0	m_9 0	m_{11} 1	m_{10} 0	
		z				
		yz				

(a) $F = yz + w'x'$

		y				
		yz	00	01	11	10
w	$w'z$	00	m_0 X	m_1 1	m_3 1	m_2 X
	01	m_4 0	m_5 X	m_7 1	m_6 0	
	11	m_{12} 0	m_{13} 0	m_{15} 1	m_{14} 0	
	10	m_8 0	m_9 0	m_{11} 1	m_{10} 0	
		z				

(b) $F = yz + w'z$

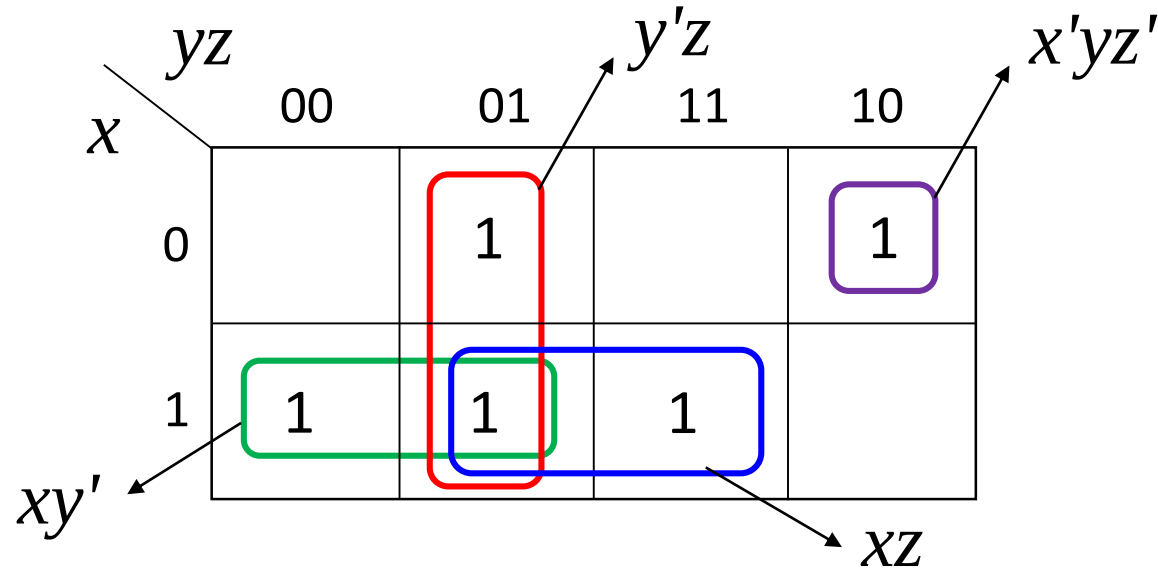
PRODUCT OF SUMS (POS) REPRESENTATION

Example: Simplify the following Boolean function into both sum of products (SOPs) and Product of sums (POSs).

i. $F = \Sigma(1, 2, 4, 5, 7)$

ii. $F = \Sigma(0, 1, 2, 5, 8, 9, 10, 13)$

Solution: i) The **SOP form** can be obtained as



$$F = xy' + xz + y'z + x'yz'$$

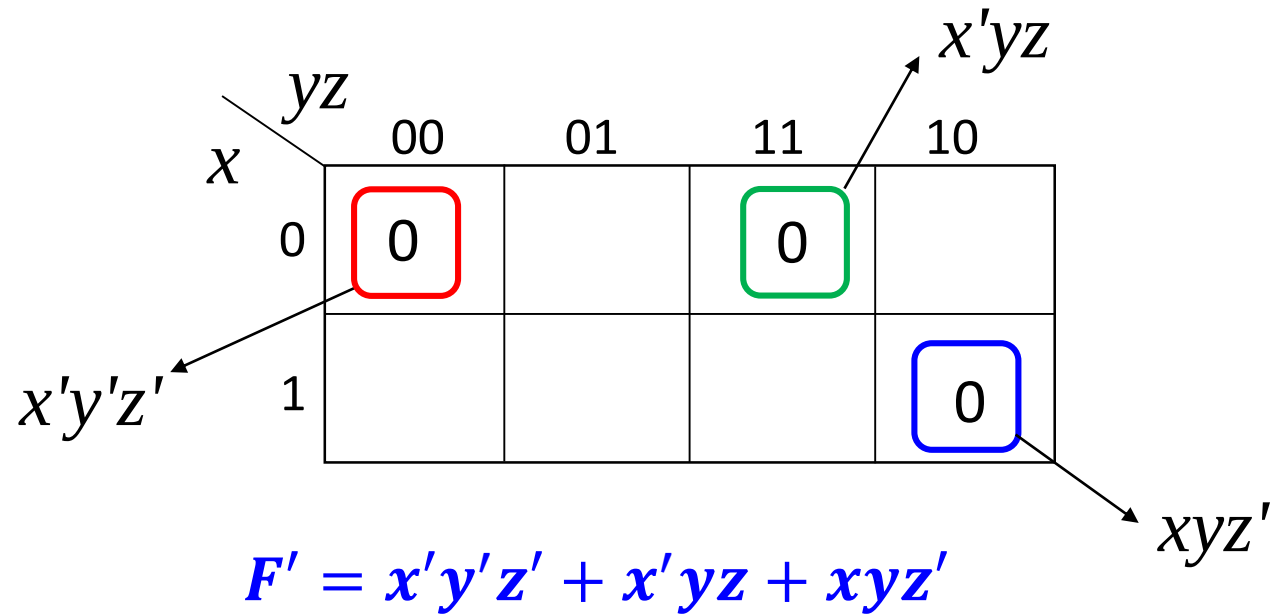
PRODUCT OF SUMS (POS) REPRESENTATION

Solution: i) The **POS form** can be obtained as

Step-1: Obtain the complement of F

$$F' = \Sigma(0, 3, 6)$$

Step-2: Minimize F' using K-map



Step-3: Obtain F using De 'Morgan's Law

$$F = (F')' = (x'y'z' + x'yz + xyz')'$$

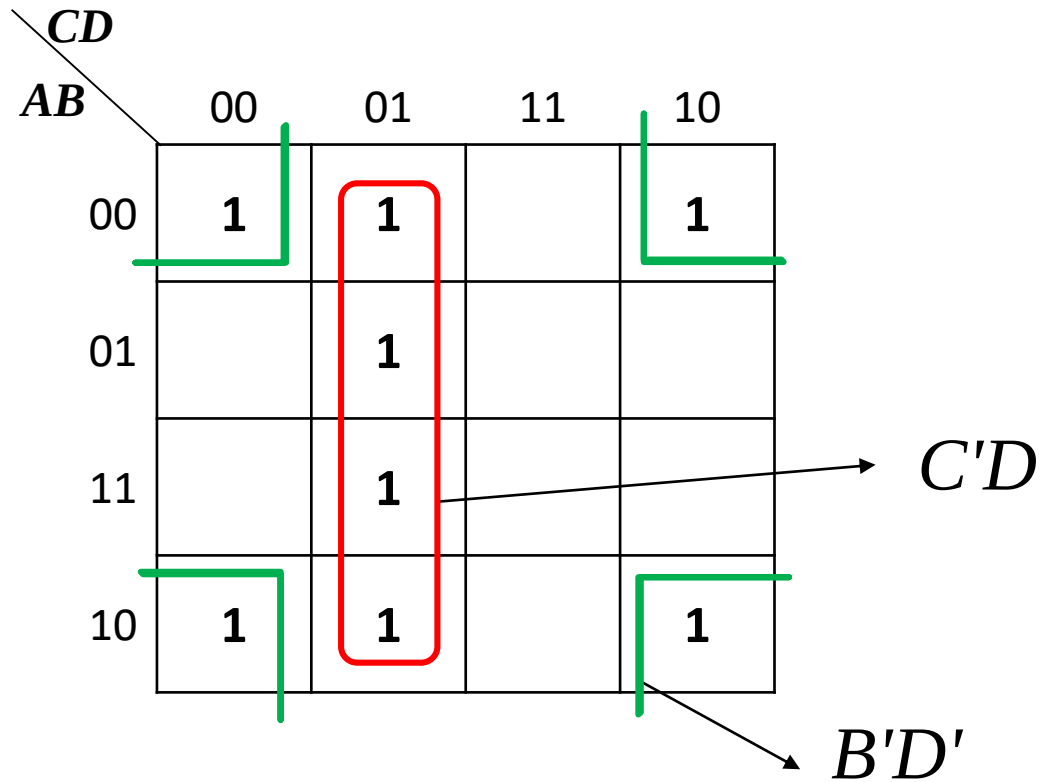
$$F = (x'y'z')'(x'yz)'(xyz')'$$

$$F = (x + y + z)(x + y' + z')(x' + y' + z)$$

PRODUCT OF SUMS (POS) REPRESENTATION

Solution: ii) The **SOP form** can be obtained as

$$F = \Sigma(0, 1, 2, 5, 8, 9, 10, 13)$$



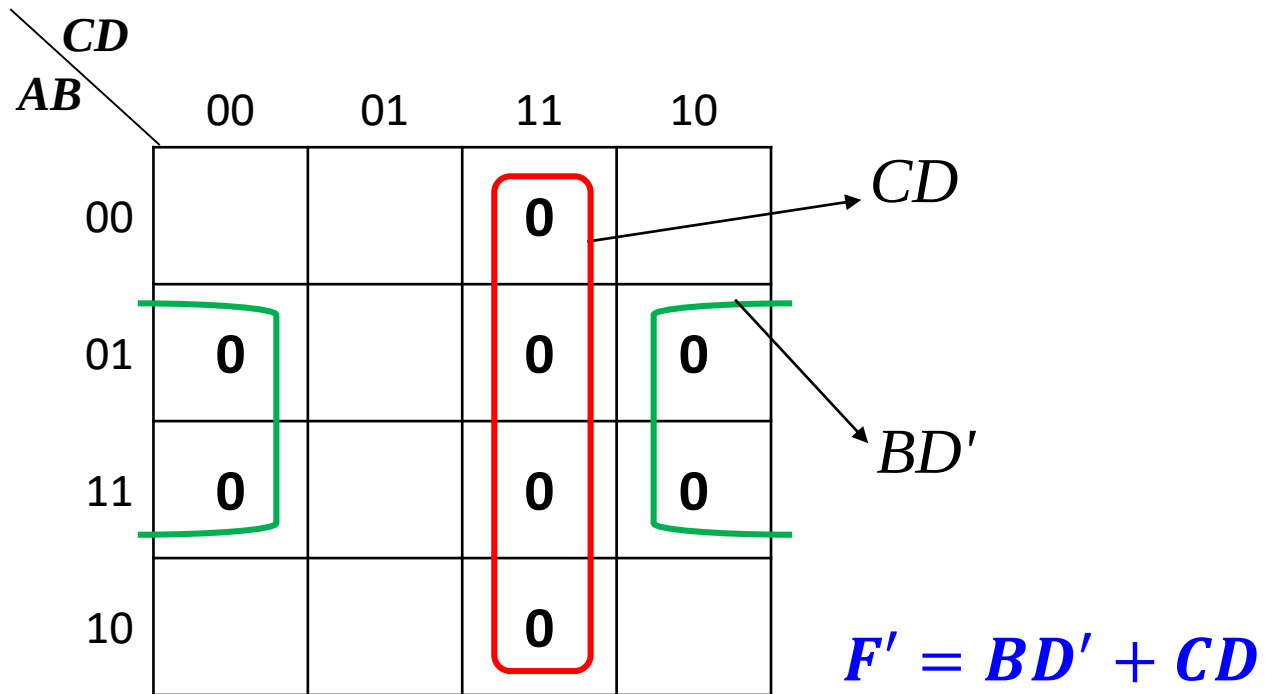
$$F = B'D' + C'D$$

PRODUCT OF SUMS (POS) REPRESENTATION

Solution: ii) The **POS form** can be obtained as

Step-1: Obtain the complement of F : $F' = \Sigma(3, 4, 6, 7, 11, 12, 14, 15)$

Step-2: Minimize F' using K-map



Step-3: Obtain F using De 'Morgan's Law

$$F = (F')' = (BD' + CD)'$$

$$F = (BD')'(CD)'$$

$$F = (B' + D)(C' + D')$$

Thank You